

Test 1 — Web (Vue 3): Fast Periodic Visual Stimulation

Ricardo Reis
Full Stack Software Engineer

11th of September, 2026

FPVS — timing write-up

What was built

A Vue 3 + Pinia SPA with three views (Setup / Run / Results). Setup configures base rate, oddball-every-N, sequence length, and stimulus set. Run presents the stream and logs the onset of every stimulus. Results reports IOI statistics, a DFT of the onset train, and exports the raw log as CSV or JSON. Stimuli are 512×512 PNGs rendered at 400×400 on a mid-grey field; base order is randomised with no immediate repeats, oddball every 5th position.

How timing was controlled

Three decisions carry almost all of the accuracy.

Precompute the schedule, do not accumulate. At run start, target onsets are computed once as absolute times: $\text{target}_k = t + k \cdot \text{periodMs}$. The loop never does $t += \text{period}$ — floating-point accumulation would drift, and more importantly it would decouple our schedule from the browser's frame clock. Because t is a single `performance.now()` reading taken 2 s before the first stimulus, every target is on the same lattice.

Preload and pre-decode everything. All stimuli are `<img>` elements, absolutely stacked in a 400×400 stage, loaded and `decode()`d before the run begins. The “presentation” is a `classList` toggle between two already-decoded, already-composited layers. There is no network, no decode, and no Vue reactivity on the hot path — the `<script setup>` body holds `schedule`, `targets`, and `log` as plain module-level values, not refs, so `tick()` is indexing arrays and toggling a class. That is the fastest thing you can do to a DOM element short of WebGL.

One `requestAnimationFrame` loop, catch-up semantics. On each frame, the loop presents every stimulus whose target time has already passed, with a 1 ms tolerance to absorb FP jitter between our t and the browser's frame clock. If more than one target is due in a frame (because the tab stalled), it presents the latest and logs the earlier ones as missed — the log always records the truth rather than fabricating onsets.

The onset time recorded is `performance.now()` read immediately after the class toggle, relative to t . Each entry stores the target time, the measured onset, the deviation ($\text{onset} - \text{target}$), the stimulus id, and whether it was an oddball. That deviation column is the whole point: it lets Results show drift and jitter separately, not just their aggregate.

How accurate it turned out

Mean IOI: 166.657 ms against a target of 166.667 ms → realized 6.00034 Hz, i.e. +0.006 % of nominal (+0.00034 Hz). The mean is essentially exact because the target grid is exact; the residual is the average of a zero-mean jitter plus a small rate offset (below).

SD of IOI: 0.143 ms. This is the number that actually limits the paradigm. At 6 Hz on a 60 Hz display the period is exactly 10 frames, so the theoretical floor from frame quantisation alone is **0 ms** — every target sits on a frame boundary. The 0.143 ms we observe is therefore browser scheduling jitter, not clock error, and it is far below the frame quantum: `performance.now()` inside the rAF callback resolves sub-frame, so onsets land with sub-millisecond precision on either side of the target.

Min / max IOI: 165.5 / 167.8 ms — a spread of 2.30 ms over 359 intervals. That is tight enough that a photodiode-free analysis would not distinguish this run from a perfect one.

Worst IOI vs target: 1.167 ms — the single worst interval was off by about a millisecond, which is well within one frame.

Worst absolute schedule offset: 3.900 ms — the first stimulus of the run, where the offset between our t and the frame boundary the rAF loop landed on is largest. After that the offset decays monotonically: 3.90 ms at onset 1, 2.73 ms at onset 50, 2.30 ms at 100, 1.67 ms at 200, 1.00 ms at 300, 0.47 ms at onset 360. The decay is a slow drift of the stimulus lattice *relative to the ideal target grid*, not jitter, and it is exactly accounted for by the difference between the target period (166.6667 ms) and the measured mean IOI (166.6571 ms): 0.0096 ms per interval, 3.43 ms over 360 intervals. In relative terms that is 5.7×10^{-5} — the same number the `rateErrorPct` field reports, so it is one fact, not two. The IOI SD of 0.143 ms is measured about the mean and is unaffected by the drift; it captures only jitter.

Spectrum. The onset DFT peaks at 6 Hz with 357.4× prominence over the median of the spectrum, and its harmonics at 12 and 18 Hz are visible and clean — the stream was periodic. The oddball-weighted DFT (weights +1 for oddballs, -1 for base) peaks at 1.2 Hz with 70.0× prominence, and its 5th harmonic ($5 \times 1.2 = 6.0$ Hz) lands exactly under the base rate, which is arithmetic proof that the every-5th cadence survived the run and not just the timing. The 1.2 Hz prominence is smaller than the 6 Hz onset prominence, as expected: only one in five stimuli carries the oddball weight. The weighted spectrum has additional peaks at 2.4, 3.6, 4.8, 7.2 Hz, which are the higher harmonics of the 1.2 Hz impulse train — the expected comb, not an artefact.

What limits the precision you can actually prove

The measurement stops at the DOM boundary. `performance.now()` fires when our class toggle returns; the compositor then paints, and the display panel responds some tens of milliseconds later. Nothing accessible from JavaScript observes the actual pixel flip. A photodiode on real hardware is the only instrument that closes that loop, and it is not available in a browser. So the strongest claim we can make is **“application-layer onset tracked the target grid to within 1.17 ms of any individual target, with a measured rate error of 5.7×10^{-5} ”** — not “the stimulus was on screen at time t ”. The drift visible in the deviation plot is the size of the mismatch between the ideal clock and the actual display clock; the gap between our code running and the pixel flipping is unknown and is not measured by anything in this app.

The browser’s frame scheduler is not real-time. A 60 Hz display gives a 16.67 ms quantum; at 6 Hz that happens to divide evenly, which is why the SD can approach the sub-millisecond floor here. At a non-divisor rate — 6.5 Hz is 9.23 frames — the IOIs must alternate 9 and 10 frames, and the SD floor rises to roughly 8 ms no matter what the code does. The rate and the display together set the floor; the code only determines how close you get to it.

Background tabs and the main thread are hostile. A GC pause, a layout on another tab, or the browser deprioritising rAF while the window is unfocused will drop frames. The Run view aborts if it sees a >500 ms gap between frames rather than silently emitting a burst of fake onsets, and Results flags partial runs so the statistics are not read as if the run completed.

Variable-refresh displays have no fixed lattice. On ProMotion or VRR, the frame period itself changes; there is nothing stable to snap to, and the honest move is to fall back to the wall-clock schedule and say so in Results. This run was captured on a fixed-rate display; the same code on a VRR panel would report a higher SD floor through no fault of its own.

Placeholder stimuli are not experimental stimuli. The set here is procedurally generated colour gratings; the render path is identical, but the content-level claims a real FPVS study would make do not apply.

Next steps

If this were going further:

- measure the refresh rate at startup and snap `periodMs` to an integer number of frames so the schedule is guaranteed to sit on the frame lattice rather than hoping it does;
- ship stimuli at 400×400 rather than 512×512 to remove a resample;
- include Quasar components, because the look is slightly 1993;
- and — the only way to actually close the loop — validate against a photodiode on real hardware. The 5.7×10^{-5} rate error is small enough that a 60 s run at 6 Hz stays within one frame of the target grid throughout, but it would matter for a longer run or for averaging across many runs, which is precisely why the refresh rate should be measured and the period snapped to an integer number of frames, so the lattice and the display agree by construction rather than by luck. Until then, the browser ceiling is what the numbers above show, and the write-up should say so plainly.